

JWT Authentication System Design

Table of Contents

1. Introduction
 2. System Requirements
 3. Why Not Session-Based Authentication?
 4. Problems with Sessions
 5. JWT-Based Solution
 6. Login Flow
 7. JWT Structure
 8. Token Storage Strategies
 9. Accessing APIs
 10. What is a Bearer Token?
 11. API Gateway Verification Flow
 12. Microservice Authentication Flow
 13. Access Token Lifecycle
 14. Refresh Token Lifecycle
 15. Refresh Flow
 16. Refresh Token Rotation
 17. Logout in JWT Systems
 18. Complete JWT Architecture
 19. Interview Conclusion
 20. Key Takeaways
-

1. Introduction

Modern applications such as Netflix, Amazon, Uber, Spotify, Swiggy, and Zomato serve millions of users simultaneously.

JWT (JSON Web Token) is commonly used because it allows authentication to be stateless, scalable, and suitable for distributed systems.

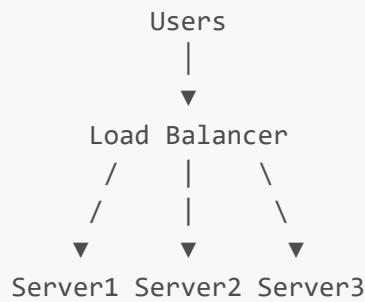
2. System Requirements

Requirements:

- Millions of users
- Multiple backend servers
- Mobile applications
- Web applications
- High API traffic
- Horizontal scalability

- Secure authentication

Architecture:



3. Why Not Session-Based Authentication?

Session-based authentication stores user information on the server.

Example:

```
Session ID = S123
```

```
S123 → User42
```

Problem:

If a request goes to another server that does not have the session, the user may appear logged out.

4. Problems with Sessions

Sticky Sessions

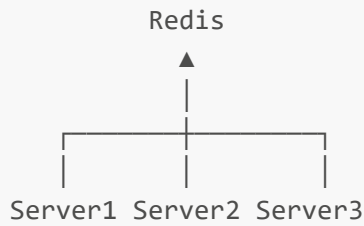
Always route a user to the same server.

Problem:

```
Server crashes  
↓  
All sessions lost
```

Shared Session Store

Store sessions in Redis.

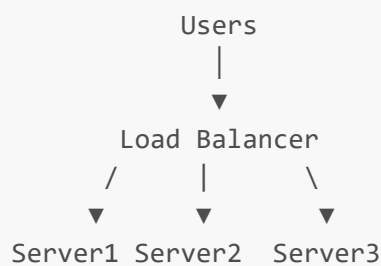


Drawbacks:

- Additional network calls
- Increased latency
- Higher infrastructure cost

5. JWT-Based Solution

JWT removes server-side session lookups.



Any server can validate the token independently.

Benefits:

- Stateless authentication
- Better scalability
- Microservice friendly

6. Login Flow

Step 1: User Login

```
POST /login
```

```
{  
  "email": "alice@gmail.com",
```

```
"password": "123"
}
```

Step 2: Verify Credentials

```
Receive Credentials
  ↓
Query User Database
  ↓
Verify Password
```

Example:

```
bcrypt.compare(
  plainPassword,
  storedHash
)
```

Step 3: Generate Tokens

Access Token

```
{
  "sub": "42",
  "email": "alice@gmail.com",
  "role": "USER",
  "iat": 1710000000,
  "exp": 1710000900
}
```

Expiry:

15 Minutes

Refresh Token

```
{
  "sub": "42",
  "tokenId": "RT123",
  "exp": 1710600000
}
```

Expiry:

7 Days

7. JWT Structure

JWT consists of:

HEADER . PAYLOAD . SIGNATURE

Header

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Payload

```
{  
  "sub": "42",  
  "role": "USER",  
  "exp": 1710000900  
}
```

Signature

```
HMACSHA256(  
  Base64Url(Header)  
  +  
  "."  
  +  
  Base64Url(Payload),  
  SECRET_KEY  
)
```

Final JWT:

```
header.payload.signature
```

8. Token Storage Strategies

Recommended:

```
Access Token → HttpOnly Cookie  
Refresh Token → HttpOnly Cookie
```

Alternative:

```
Access Token → Memory  
Refresh Token → HttpOnly Cookie
```

Avoid:

```
localStorage
```

Reason:

```
XSS attacks can steal tokens.
```

9. Accessing APIs

```
GET /orders  
Authorization: Bearer eyJ...
```

10. What is a Bearer Token?

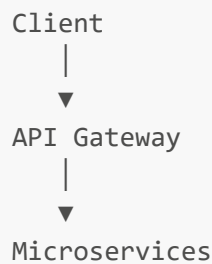
Bearer means:

Whoever possesses the token gets access.

Therefore HTTPS is mandatory.

11. API Gateway Verification Flow

Architecture:



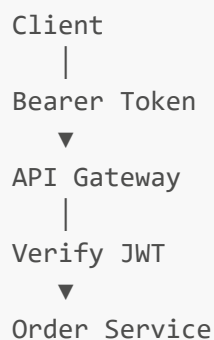
Verification Steps:

1. Extract token
2. Decode header
3. Decode payload
4. Verify signature
5. Verify expiration
6. Create authenticated user context

If invalid:

401 Unauthorized

12. Microservice Authentication Flow



▼
Database

13. Access Token Lifecycle

09:00 Login
09:15 Token Expired

Flow:

```
graph TD
    A[Login] --> B[Receive Access Token]
    B --> C[Call APIs]
    C --> D[Token Expires]
    D --> E[Refresh Required]
```

14. Refresh Token Lifecycle

Day 1 Login
Day 7 Refresh Token Expired

15. Refresh Flow

POST /refresh

Cookie: RefreshToken

Server:

```
Validate Refresh Token
↓
Generate New Access Token
↓
Return New Access Token
```

16. Refresh Token Rotation

```
RT1 → RT2 → RT3 → RT4
```

Each refresh:

```
Old Refresh Token
↓
Invalidate Old Token
↓
Issue New Refresh Token
```

Replay attack detection:

```
Replay Attack
↓
Terminate Session
```

17. Logout in JWT Systems

Problem:

```
Access Token may still be valid.
```

Solutions:

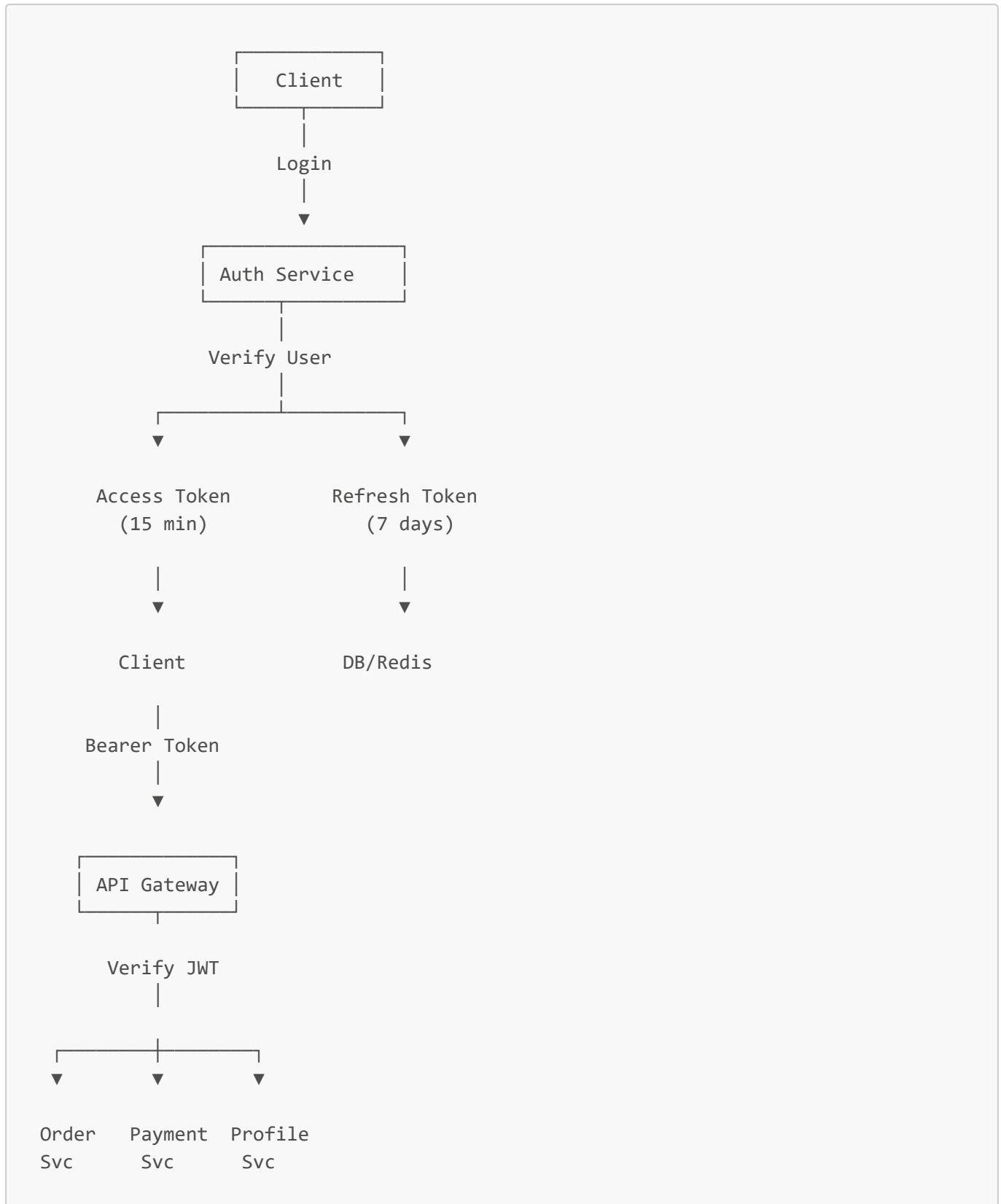
Short Access Tokens

```
15 Minutes
```

Refresh Token Revocation

RT123 → Revoked

18. Complete JWT Architecture



19. Interview Conclusion

JWT-based authentication is preferred in distributed systems because authentication becomes stateless.

Any server can validate the token locally without querying a centralized session store.

Short-lived access tokens provide scalability and security, while long-lived refresh tokens improve user experience by allowing token renewal without requiring users to log in repeatedly.

20. Key Takeaways

- JWT stands for JSON Web Token.
- JWT authentication is stateless.
- No centralized session lookup is required.
- Any server can validate JWTs independently.
- Access Tokens should be short-lived.
- Refresh Tokens should be long-lived.
- HTTPS is mandatory.
- API Gateways commonly perform JWT validation.
- JWT works extremely well in microservices and distributed systems.
- Refresh token rotation improves security.